PRODUCT & ENGINEERING SPEC · SYNDICATE BY MAXIMOR — TRACK 2

Ledger Sense

A blank-slate build spec for an exception-resolution memory in finance operations: it matches transactions through a cost-tiered pipeline, routes what it can't close to the right owner with a live SLA clock, blocks or holds anything policy says shouldn't move, and — the part that has to carry the demo — learns from how a human resolves an exception so the same class of exception stops recurring.

Spec status complete — ready to implement, nothing assumed pre-built **Target build window** ~30h, remote solo

Reference dataset seed=42, 26,738 bank lines (§4) **Owner** himanshuthakur8119@gmail.com

---

Finance software has already automated the known, structured cases. What's left is ambiguous work: deciding whether two records describe the same event, figuring out who actually owns a weird exception, and reconstructing context that lives in someone's head, not a database. Matching engines already exist. Routing/workflow tools already exist. What this spec builds is the piece that doesn't: a system that **closes the loop** — turns this month's human judgment call into next month's automatic resolution, for that specific class of problem, provably.

Five agents, each doing something a single prompt can't fake because each owns a genuinely different tool and a genuinely different slice of the data. None of them exist yet. This document specifies all five in enough detail — schemas, thresholds, formulas, acceptance criteria — to implement directly, in any order that respects the dependency chain in **§13**, without needing to read any other document first.

§1 Positioning — say this, not that

The trap to avoid: describe this as "AI reconciliation" and a sharp judge files it next to BlackLine and FloQast on the spot, both of which already do transaction matching. The five agents below don't need to change to avoid that trap — what needs to be said differently is what they're for.

DON'T PITCH IT THIS WAY

"An autonomous agent that reconciles transactions."

PITCH IT THIS WAY

"A system that captures how your team resolves the exceptions no rule can close — and retires that exception class."

> **Finance software automates known rules. Ledger Sense learns the organization's recurring way of resolving the exceptions those rules can't handle.**

Two build consequences follow. First, Agent 1's match rate is table stakes to demo, not the headline — don't spend the demo's best minutes on it. Second, Agent 3 (Resolution-Learning) cannot ship as "captures a resolution and stores it" — it must be able to say, honestly, whether it learned a class of exception or just memoized one transaction. §7 is written to force that distinction into the build.

§2 Why a CFO — not just a controller — should care

The pain this system attacks is mostly felt by a controller first: chasing an AP reference, reconciling a bank fee, waiting on a warehouse count. A CFO doesn't spend their day matching line items. But the pain becomes CFO-relevant through a specific, short mechanism — say it explicitly in the README and the demo rather than assuming a judge connects the dots:

LINK	WHAT HAPPENS	WHO FEELS IT
1	An exception has no rule that closes it	Controller / recon ops
2	It waits on a human owner and an SLA clock	Controller, AP/AR
3	Close is delayed until the queue clears	Controller → Finance team
4	Forecast is built on last month's unresolved numbers	FP&A
5	Confidence in the reported number drops	**CFO**

Every rule Agent 3 promotes shortens step 2 — the only step in that chain a piece of software can actually shrink. **Straight-through rate is the one number in this whole chain that maps directly onto close-cycle days**, which is the CFO-legible translation of "we did agent engineering well."

§3 System overview

Five agents, one shared data spine. Each reads the previous stage's output files and writes its own — no agent imports another agent's internals, and no agent may read the ground-truth file described in §4. That file boundary is what makes "five real agents" true rather than five functions in one process.

#	AGENT	ANSWERS	READS	WRITES
1	**Matching**	Did this bank line reconcile against a ledger entry?	`ledger.csv`, `bank.csv`	`match_outcomes.csv`, `ledger_settlements.csv`
2	**Ownership / Routing**	Whose problem is this, and by when?	Agent 1's output	`exceptions.csv`, `owner_queues.csv`
3	**Resolution-Learning**	How did a human just resolve this, and does it generalize?	Agent 2's output + a human resolution	`rules.json`
4	**Escalation / Guardrail**	Is this payment allowed to release?	`ledger.csv`, `bank.csv`, Agents 1–2's output	`release_decisions.csv`, `guardrail_audit.csv`
5	**Metrics Orchestrator**	Did straight-through rate actually climb, and why?	Two full passes through Agents 1–4	the live scoreboard

`bank.csv + ledger.csv` → `Agent 1 · match_outcomes.csv` → `Agent 2 · exceptions.csv` →
`Agent 3 · rules.json` ↻ back into Agent 1's pass-2 run

`ledger.csv + bank.csv + Agents 1–2` → `Agent 4 · release_decisions.csv`

Agent 4 runs alongside Agents 1–2, not after them — it re-derives some of what they found (see §8's `duplicate_release` rule) on purpose, as an independent check, and it consumes their output only for corroboration and for carrying forward anything they already flagged. Agent 5 doesn't produce a new CSV; it runs the whole left-hand chain twice and reports the delta.

§4 Data model & synthetic batch generator

Every agent below depends on this section existing first. Build it first, and validate the whole spec's numeric targets against it before touching Agent 1 — everything downstream cites this exact dataset.

HOW TO READ THE NUMBERS IN THIS DOCUMENT

Every threshold and percentage figure in §5–§9 (cheap-tier rate, straight-through rate, block/hold split, etc.) is a **target**, calibrated once against the specific 26,738-line dataset this section defines (`seed=42`, `25,000` cases, pass 1). They tell you what "the design is working" looks like on this exact data — treat them as acceptance thresholds to build toward and regression-pin, not as a report on work already done.

4.1 · The two sides, plus ground truth

TABLE	FILE	MEANING
`LedgerEntry`	`ledger.csv`	What our books say happened. Clean, structured, authoritative on intent.
`BankTransaction`	`bank.csv`	What the bank/PSP statement reports. Free-text, mangled references, split or duplicated postings.
`MatchLink`	`match_links.csv`	**Ground truth, not input.** Which (ledger, bank) pairs should be linked, and what mess was injected.

HARD RULE

Agents 1, 2 and 4 must never read `match_links.csv`. It exists so Agent 5 can compute straight-through rate, precision and recall, and so a human can score Agent 2's category choices after the fact. Enforce this with a grep test over each agent's package plus an AST-level import check — a string search alone misses `outcome.score`-style access.

`LedgerEntry` — `ledger.csv`

COLUMN	TYPE	NOTES
ledger_id	str	PK, `LG-P1-000042`. Unique in batch.
booked_at	ISO-8601 UTC	When we booked it.
amount	decimal, 2dp	**Signed.** Positive = money we expect in; negative = money out.
currency	str	ISO-4217, always clean uppercase on this side.
entry_type	enum	`invoice_payment`, `subscription_charge`, `refund`, `payout`, `fee`, `adjustment`.
counterparty_id	str	`CP-0231`.
counterparty_name	str	Canonical name. Clean.
reference	str	`INV-2026-1000042`. What we quoted.
memo	str	Human description, consistent with `entry_type`.
account_code	str	`1200` (AR) for inflows, `2100` (AP) for outflows.
source_system	str	`billing` / `payouts` / `manual`.

`BankTransaction` — `bank.csv`

COLUMN	TYPE	NOTES
bank_txn_id	str	PK, `BK-P1-000117`. Unique in batch.
value_date	ISO-8601 UTC	May precede or badly lag `booked_at`.
amount	decimal, 2dp	Signed, same convention. May be 0, sign-flipped, partial or fee-skewed.
currency	str	ISO-4217 but **not guaranteed** uppercase or unpadded.
counterparty_name_raw	str	Statement name. Noisy by default (§4.3).
reference_raw	str	May be empty, wrong, or junk-separated.

COLUMN	TYPE	NOTES
description	str	Full narrative, e.g. `ACH CREDIT ACME LOGISTICS PYMT REF INV-2026-1000042`.
bank_account	str	`ACCT-USD-01`.
statement_id	str	`STMT-P1-006` (one per week of the span).
direction	str	`credit` / `debit`, derived from sign.

`MatchLink` — `match_links.csv` (ground truth)

COLUMN	TYPE	NOTES
ledger_id	str	FK.
bank_txn_id	str	FK.
relation	enum	`exact`, `partial`, `duplicate`.
defect	enum	The injected mess — full taxonomy in §4.2.
case_id	str	Groups all records from one economic event (`C-P1-000042`). A partial payment yields two links sharing a `case_id`.
note	str	Human-readable explanation — "why did case #47 escalate."

Cardinality: a case is normally 1 ledger : 1 bank. Duplicates and two-part partial payments are 1 : 2. Orphans are 1 : 0 or 0 : 1. **Money is `Decimal` in memory, a fixed 2-decimal string in CSV — never a float.**

4.2 · The messy-case taxonomy

DEFECT	RATE	WHAT THE DATA LOOKS LIKE
clean	57.0%	Reference exact, amount equal, dates within ~2 days. Name still noisy.
wrong_reference	7.0%	Bank quotes another ledger entry's reference.
partial_payment	6.0%	One ledger entry settled by one or two smaller bank postings.
out_of_order	6.0%	Value date precedes booking by 1–6 days, or lags by 21–48 days.
duplicate	5.0%	Same event posted twice, different `bank_txn_id`.
missing_reference	5.0%	`reference_raw` empty.
fx_rounding	4.0%	Amount off by 0.01–3.50.
malformed	2.5%	Padded lowercase currency/name, junk reference separators.
negative_amount	2.0%	Sign flipped. **Guardrail bait.**
orphan_bank	2.0%	Bank record, no ledger entry at all.
orphan_ledger	2.0%	Ledger entry, never settled.
zero_amount	1.5%	`0.00` posting. **Guardrail bait.**

Baseline name noise (not a defect): 85% of bank records get a perturbed counterparty name — uppercased, suffix dropped, truncated, abbreviated. This is what makes fuzzy matching in Agent 1 earn its place over plain string equality.

4.3 · Determinism and the two demo passes

The single most important design constraint: `(seed, pass_number, n_cases)` **must determine the output byte-for-byte.** Two RNG streams:

- **Counterparty universe** — seeded from `seed` only. Same ~800 customers across passes, so a rule learned in pass 1 is still applicable in pass 2.

- **Case stream** — seeded from `(seed, pass_number)`. Pass 2 has different transactions, identical statistical shape, non-overlapping references.

> NON-NEGOTIABLE
>
> Pass 2 must not be easier data than pass 1. Any straight-through-rate climb has to come entirely from Agent 3's learned rules.

§5 Agent 1 — Matching

FIRST AGENT TO BUILD · DEPENDS ONLY ON §4

Answers one question per bank line: does this reconcile against a ledger entry, and how confidently? A cheap deterministic tier resolves the clear majority for zero LLM calls; only the honest residual escalates.

5.1 · Why two tiers

Build the cheap tier as five scored features and a weighted sum; on §4's dataset it should clear roughly **85–90%** of the batch on its own. The LLM tier should only see lines where deterministic evidence genuinely conflicts — chiefly `wrong_reference`.

5.2 · Candidate generation (blocking)

INDEX	KEY	CATCHES
by_ref	normalized reference	Everything with a usable reference.
by_key4	first 4 chars of squashed name	Name-only evidence, never used unintersected.
by_bucket	`abs(cents) // 100`	Amount-only evidence.

`candidates()` = union of (1) exact reference, (2) `by_key4` ∩ ±4 bucket window, (3) — only if 1&2 empty — `by_key4` filtered to a plausible part-payment ratio. Cap at 40 candidates, keep closest amounts on overflow. **No full-scan fallback** — an unblocked line is correctly `no_candidate`.

A name-based block key is only safe if every noise variant preserves the first 4 alphanumeric characters of the canonical name — assert this as a generator invariant.

5.3 · Scoring — five features, 0–100

FEATURE	WEIGHT	VALUES
reference	40	1.0 equal · 0.6 fuzzy≥90 · 0.0 different · dropped if no reference
amount	30	exact 1.0 · fx 0.8 · partial 0.55 · conflict 0.0
name	20	max(token_set_ratio, partial_ratio)/100, floored at 0.40
date	7	1.0 within ±3 days, decay to 0 at ±60
currency	3	1.0 equal, else 0.0

Renormalization rule: no reference → drop that feature, renormalize the rest to 100 (amount 50, name 33.3, date 11.7, currency 5) — missing evidence isn't counter-evidence. A wrong reference scores 0.0 but keeps its 40-point weight, so the true counterpart lands ~78 and is refused — the mechanism that sends `wrong_reference` to the LLM tier.

Short-circuit: `reference==1.0 && amount==exact && currency==1.0` → 100 immediately, skipping fuzzy work — roughly two-thirds of the batch.

Amount classes (integer cents, never float): `exact` same sign/cent; `fx` delta ≤ max(3.50, 0.5%) and ≤¼ of entry; `partial` 15–85% of entry; `conflict` everything else.

5.4 · The acceptance rule

Auto-match when: no guardrail-interlock veto, `score≥88.0`, `margin≥6.0` (or single candidate), capacity remains. Plus `PARTIAL_WITH_EXACT_REFERENCE`: accept at `score≥78.0` when reference exact, amount=partial, capacity remains, `name≥0.70`. Escalate 45–88; reject below 45.

> KNOWN LIMITATION TO DESIGN IN FROM THE START
>
> A near-relative decoy name (e.g. "Alpha Systems Group" vs "Alpha Systems Trading Co", ~0.84 fuzzy) can slip through the partial exception on reference credit alone. Real fix needs the runner-up's own amount+name score weighed, not just the top candidate's blend — a scoring-design change. Ship as a documented, regression-pinned known limitation.

5.5 · Capacity, duplicates and partials

Greedy assignment in descending `(score, bank_txn_id)` order against one capacity ledger in cents — both tiers settle through it, so an LLM accept can't double-book. Duplicate: first leg wins, second finds zero capacity and is emitted `duplicate_of_matched` with `matched_amount=0.00`. Partial: each leg its own row; book side gets a settlement with a residual.

5.6 · The guardrail interlock

A two-condition veto inside Agent 1 (separate from Agent 4): 0.00-vs-nonzero, flipped sign, or currency mismatch on the best candidate — never auto-matched by either tier, re-checked after an LLM verdict too.

5.7 · The LLM seam

An adjudicator protocol: one batched call per run, each question carrying the bank record, top-3 candidates with per-feature breakdown, and the cheap tier's note on why it couldn't close. Ship a zero-cost stub (flagged `llm_is_stub=True`) and a `none` adjudicator so `llm_calls==0` is measured, not asserted. A real provider (§10) is one new adjudicator class.

5.8 · Output contract

```
match_outcomes.csv   one row per bank transaction, always
  bank_txn_id, status, relation, ledger_id, tier,
  score, margin, reason, reason_detail,
  matched_amount, residual_after, candidates, features,
  llm_model, llm_confidence, llm_is_stub

ledger_settlements.csv   one row per ledger entry, always
  ledger_id, ledger_amount, matched_amount, residual,
  n_parts, bank_txn_ids, fully_settled, reason
```

5.9 · Acceptance targets, against §4's reference dataset

TARGET	VALUE
Cheap-tier rate (zero LLM calls)	~88–89%
Overall match rate, with stub adjudicator	~92–93%
Precision of matched rows vs. ground truth	≥ 0.999
Duplicates flagged, never double-settled	100%
Guardrail bait auto-matched	0
Orphan bank lines matched	0
Two runs, identical input	byte-identical CSVs

§6 Agent 2 — Ownership / Routing

DEPENDS ON §4 AND §5

Everything Agent 1 couldn't close arrives here as a residual and must leave as work: a category, a named owner, and a deadline that's a pure function of an explicit "now." Routing well means routing little.

6.1 · Five categories, not fifteen

CATEGORY	WHAT IT MEANS	DESK	BASE SLA
duplicate	Same event posted twice; correct action is to void.	recon ops	24 h
amount_mismatch	We know who; we disagree how much.	AR / AP	48 h
timing	Money and party agree; the calendar doesn't.	AR / AP	120 h
unidentified_counterpart	We cannot say whose money this is.	AR / AP	72 h
suspect_posting	The posting itself isn't credible.	controller	4 h

`suspect_posting` earns a fifth category: different question (book-integrity, a controller's call), different clock (always P1 at 4h regardless of amount), different consumer (Agent 4 picks these up for an approval chain).

6.2 · The bank-side classifier — ordered, first hit wins

#	CONDITION	CATEGORY
1	`reason ∈ {anomalous_amount, currency_conflict}`	`suspect_posting`
2	`relation is duplicate`	`duplicate`
3	`reason is ledger_already_settled`	`amount_mismatch`
4	`amt==conflict` & `name≥0.70`	`amount_mismatch`
5	`date<0.50` & `name≥0.70` & `amt∈{exact,fx}`	`timing`
6	`amt==partial` & `name≥0.70`	`timing`
7	everything else	`unidentified_counterpart`

Use the same `0.70` name floor as §5.4 — a routing rule claiming "we know the counterparty" must never be more generous than the matching rule making the same claim. Book side: `never_settled` → `timing` ; `partially_settled` residual ≥15%→ `timing` ; <15%→ `amount_mismatch` (reuse Agent 1's 15–85% partial band).

6.3 · Pair-and-suppress

A bank subject whose top candidate is itself an unclaimed ledger subject is emitted once, as `subject_kind=pair` — the `wrong_reference` shape. Ties break by `bank_txn_id` order.

6.4 · Ownership — a person, never a queue

Resolve to one named individual on a fixed roster (e.g. eleven people: 3 AR / 3 AP / 3 recon-ops / 2 controllers). Desk: `suspect_posting→controller` , `duplicate→recon_ops` , else AR if inbound else AP. Individual: hash the counterparty (not the transaction) with **`blake2b`** , **never** `hash()` — Python's built-in is per-process salted and would silently reshuffle owners between runs. No capacity/spill logic — report load instead.

6.5 · The SLA clock

`sla_hours = BASE_SLA_HOURS[category] × SEVERITY_MULTIPLIER[severity]` , P1×0.5 / P2×1.0 / P3×1.5. `suspect_posting` always P1. Otherwise bucket on `abs(amount)` in rough USD: ≥10,000→P1, ≥1,000→P2, else P3. Clock starts at the run's explicit `as_of` — never `datetime.now()` . `clock(opened_at, due_at, now)` is a pure function; `at_risk` below 25% remaining, `breached` at `now≥due_at` . Plain elapsed hours, no business-hour calendar (documented simplification).

6.6 · Collision safety

1. Distinct identity — `exception_id` uniqueness enforced, hard error on collision.
2. Independent clocks per exception.
3. Contiguous per-owner queue positions, sorted by `(due_at, exception_id)`.
4. Owner queue counts reconcile exactly to the row list.

6.7 · Output contract

```
exceptions.csv   one row per routed subject; straight-through = no row
  exception_id, pass_id, subject_kind, bank_txn_id, ledger_id,
  category, classification_detail, match_status, match_reason,
  settlement_reason, counterparty_key, counterparty_label,
  amount, currency, severity, owner_id, owner_name, owner_team,
  assignment_basis, opened_at, sla_hours, due_at,
  hours_remaining, sla_state, sla_display, queue_position,
  age_days, evidence

owner_queues.csv   one row per roster member, incl. empty desks
  owner_id, owner_name, owner_team, open_exceptions,
  n_p1, n_p2, n_p3, earliest_due_at, n_breached
```

6.8 · Acceptance targets, against §4's reference dataset

TARGET	VALUE
Straight-through rate	~87–88%
Routed exceptions with category/owner/clock	100%
Duplicates (ground truth) → `duplicate`	100%
Guardrail bait → `suspect_posting`	≥ 95%
Two runs, identical input	byte-identical CSVs

§7 Agent 3 — Resolution-Learning

THE CORE BET · DEPENDS ON §5, §6 AND §8'S POLICY BOOK

Captures how a human resolves a routed exception and turns it into a rule, so that class of exception auto-resolves next time. Design it first, build it last of the four data agents.

WHAT A HUMAN ACTUALLY RECORDS

7.1 · Structured resolution capture

```
resolution {
  exception_id      → the exceptions.csv row this closes
  resolution_type   → fee_offset | reference_transform |
                      counterparty_alias | timing_tolerance |
                      manual_one_off | no_pattern
  evidence          → the feature-space predicate (§7.2)
  rationale         → free text, audit + demo narration only
  resolved_by, resolved_at
}
```

`manual_one_off` and `no_pattern` are first-class outcomes, not failures.

WHERE THE KNOWLEDGE LIVES, AND WHETHER IT KILLS A CLASS

7.2 · Pattern generalization, in the matcher's own vocabulary

A learned rule is a predicate over Agent 1's own feature space — counterparty key, amount-delta bucket, reference-transform type, currency — never a new representation. Example: a $2.50 Acme shortfall resolves not as "transaction #48213 is fine" but as `counterparty≈Acme AND 0<amount_delta≤$3 AND reference matches`, tested against every other Acme line.

REPEATS ENOUGH, TRUSTED ENOUGH

7.3 · Promotion gate, and the Agent 4 interlock

Corroboration: a single-resolution rule is a candidate only, promoted on explicit human confirmation or a corroboration count. **Guardrail veto:** a promoted rule is checked against Agent 4's policy book before it may fire — it can never auto-resolve something Agent 4 would independently block or hold.

WHERE PASS 2'S NUMBER COMES FROM

7.4 · Consumption contract

Promoted rules write to `rules.json`, consulted as a cheap check between Agent 1's cheap tier and escalation to Agent 2 — an insertion, not a rewrite. Every auto-resolution stamps the rule id that produced it.

§8 Agent 4 — Escalation / Guardrail

DEPENDS ON §4; CONSUMES §5–§6'S OUTPUT FOR CORROBORATION ONLY

A deterministic policy layer deciding whether a payment may release, independent of whether it matched — the live "wow" moment.

8.1 · Five rules, two verdicts

RULE	WHAT IT CHECKS	VERDICT
denied_party	Counterparty matches a hardcoded, JSON-overridable compliance list.	block
duplicate_release	Re-detects duplicates independently of Agent 1's flag — genuine corroboration.	block
dual_control	Amount exceeds a materiality threshold (e.g. $200,000).	hold
out_of_period	Value date outside a configurable window (e.g. 30 days) of `as_of`.	hold
upstream_veto	Carries forward anything Agents 1–2 already flagged suspect.	matches upstream severity

Block vs. hold is a real distinction: block = money does not move, full stop; hold = pending a second approval. An approvals mechanism can release a `dual_control` hold but must refuse to release a `denied_party` block. Keep the whole engine

deterministic — a compliance control has to be reproducible; route any LLM involvement through an optional off-path seam that never touches the decision itself.

PRECISION TEST TO WRITE ON DAY ONE

A denied-party token must never fire on an unrelated but textually similar name — e.g. a list entry `ORBEX` must not block `ORBEXIA CORP`.

8.2 · Output contract

```
release_decisions.csv   one row per bank line, every run
  bank_txn_id, verdict (allow/hold/block), primary_rule,
  all_firing_rules, reason, upstream_context,
  required_approvals, policy_version

guardrail_audit.csv     one row per rule firing
held_settlements.csv    held population, for the approval workflow
policy_applied.json     the exact policy book this run enforced
```

8.3 · Acceptance criteria

1. **AC1** — every bank line receives exactly one release decision.
2. **AC2** — every block/hold names a rule, a reason, upstream context, and (for holds) required approvals.
3. **AC3** — every audit finding cites a rule present in the applied policy.
4. **AC4** — no blocked line is releasable by any route, including approvals — independently re-derived, not self-reported.

8.4 · Reference verdict split, against §4's dataset

reference: ~$22.6M blocked exposure · ~$61.6M held exposure · AC1–AC4 passing at 100% of the batch

§9 Agent 5 — Metrics Orchestrator

DEPENDS ON ALL FOUR AGENTS ABOVE

Runs the full chain twice and proves, with real numbers, that pass 2 is better because of what Agent 3 learned.

9.1 · What it must show

- Straight-through rate, pass 1 vs. pass 2 — from a real second run, never asserted.
- Exceptions eliminated by class, not a raw count.
- Learned-rule count, each linked to the resolution that produced it.
- Guardrail block/hold rates holding steady across passes, as a sanity check.

NON-NEGOTIABLE

Never state a pass-2 number that hasn't actually been computed from a real second run. Maximor's own unverified 98%/2% figure failed independent verification — the same discipline applies to this dashboard's own output.

§10 Sponsor integrations

TensorMux — inference gateway/routing

Pull the container, point Agent 1's adjudicator at its OpenAI-compatible endpoint — no code change beyond the base URL, since §5.7 already isolates this. Use its metrics for a real cost-per-transaction number. **Contingency:** if Docker isn't available, ship on the §5.7 stub — an environment risk to flag plainly, not a design gap.

Neatlogs — agent tracing/observability

Initialize before importing instrumented LLM libraries; wrap each agent's run function in a named span. Use across all five agents so a judge can click into "why did case #47 escalate" and see it resolve by pass 2.

Dodo Payments — real transaction data source

Use its sandbox to generate a real `bank.csv`-shaped stream, so the demo runs on at least one genuine side. Never force it into a feature that doesn't need it.

§11 Demo script

SCENE	REQUIRES
Pass 1 runs cold	Real measured split, on camera: matched / exception / blocked.
Human resolves, on camera	The §7.1 structured capture form — more than "approve."
"New pattern learned"	The candidate predicate in plain English, plus its support count.
Pass 2 runs	Same batch shape, rule store in the loop. Rate must visibly move, traceable to the rule.
Scoreboard	Pass-1-vs-pass-2, side by side (§9.1).

§12 Success metrics

- **Straight-through rate climbs pass-1 → pass-2**, measured, on the same batch shape.
- **Every promoted rule traces to one specific human resolution.**
- **Zero learned-rule auto-resolutions land on a guardrail-blocked or -held line** — independently re-derived.
- **At least one exception class — not transaction — visibly disappears** between passes.

§13 Build sequence

STEP	BUILD	NEEDS FIRST	CAN START ONCE
1	Data model & generator (§4)	nothing	immediately
2	Matching (§5)	Step 1	§4's schemas are stable
3	Routing (§6)	Steps 1–2	§5's output is stable
4	Guardrail (§8)	Step 1; reads 2–3 for corroboration only	rule design in parallel with step 3

STEP	BUILD	NEEDS FIRST	CAN START ONCE
5	Resolution-Learning (§7)	Steps 1–4	§6's exceptions + §8's policy book exist
6	Metrics Orchestrator (§9)	Steps 1–5	a full pass-1/pass-2 cycle runs end to end
7	Sponsor integrations (§10)	the seam each plugs into	TensorMux once §5.7 exists; Neatlogs anytime; Dodo anytime

§14 Explicitly not doing (for the hackathon build)

- × **Multi-currency / FX modeling.** Doesn't strengthen the learning-loop story, adds surface for no demo payoff.
- × **A CFO analytics dashboard.** Easy for any team to build in a weekend; pulls attention from the differentiated mechanism.
- × **Corroboration-count promotion for the demo.** Correct for production (§7.3); explicit confirmation is more honest on camera.
- × **Semantic "who can actually resolve this" ownership.** §6.4 assigns by category/team/portfolio, not the real dependency chain — a genuine gap, named in §15.

§15 Open questions carried forward

A **Does a single-batch demo generate enough corroboration for "class elimination," or does §4's generator need tuning?**
Check against a real run before the demo recording, not during it.

B **Is §6.4's assignment close enough to "the person who can resolve this" that the §14 gap is safe to leave open?**
Likely where a sharp judge probes hardest if the demo leans on ownership.

C **If TensorMux can't be wired in time, does the cost-tiering story need a fallback claim?**
Say whichever is true plainly in the README.